ТЕХНИЧЕСКОЕ ИССЛЕДОВАНИЕ

# Графовое тестирование: автогенерация тестов из описания системы

development_system

*Человек описывает систему как граф (текст, JSON, Lisp-DSL), поток* ***граф-тестировщик*** *порождает тест-кейсы: для целевых состояний — достижимость, для нежелательных — исключения.*

Версия: 1.0 · 14 марта 2026

Стек: Common Lisp, SBCL, OpenRouter

**Ключевые слова:** Common Lisp, Model-Based Testing, Property-Based Testing, Граф состояний, Reachability, TLA+, Fiveam, Мет

тестов

# Содержание


**1 Введение** **5**

1.1 Постановка задачи . . . 5

1.2 Контекст системы `development_system` . . . 5

1.3 Механизм загрузки потоков . . . 6

1.4 Цель исследования . . . 7

**2 Теоретические основы** **7**

2.1 Model-Based Testing (MBT) . . . 8

2.1.1 Transition Tree Method . . . 8

2.1.2 Критерии покрытия . . . 9

2.1.3 Negative Testing через граф . . . 9

2.2 Property-Based Testing . . . 9

2.2.1 QuickCheck-подход . . . 10

2.2.2 Применение к CL-потокам . . . 10

2.2.3 CL-библиотеки для PBT . . . 11

2.3 Reachability Analysis . . . 11

2.3.1 Алгоритм . . . 11

2.3.2 Safety vs Liveness Properties . . . 12

2.3.3 Нежелательные состояния в `development_system` . . . 12

2.4 TLA+ и формальная верификация . . . 12

2.4.1 Применимость к CL-потокам . . . 12

2.5 Alloy для описания графа . . . 13

2.5.1 Relational Model . . . 13
2.5.2 Применимость . . . 14
2.6 LLM как генератор тест-кейсов . . . 14
2.6.1 Prompt Engineering для извлечения тестов . . . 14
2.6.2 Проблема галлюцинаций . . . 15

**3 Граф состояний development_system** **15**
3.1 Полный граф цикла обработки сообщения . . . 15
3.2 Инварианты системы . . . 17
3.3 Переходы и триггеры . . . 18

**4 Архитектура поток-граф-тестировщик** **19**
4.1 Обзор . . . 19
4.2 Три варианта входного формата . . . 19
4.2.1 Вариант А: естественный язык . . . 19
4.2.2 Вариант Б: JSON . . . 20
4.2.3 Вариант В: Lisp DSL . . . 21
4.3 Алгоритм генерации тестов из графа . . . 23
4.4 Интеграция с существующим мастером . . . 24

**5 Реализация** **25**
5.1 Структуры данных . . . 25
5.2 Парсинг Lisp DSL . . . 26
5.3 Парсинг JSON . . . 28
5.4 Алгоритмы анализа графа . . . 29
5.5 Выполнение тестов . . . 31

5.6 Точка входа потока . . . 33

**6 Property-Based и Contract Testing** **35**
6.1 Fiveam: базовое тестирование . . . 35
6.2 Property-Based Testing с check-it . . . 36
6.3 Contract-Based Testing . . . 38
6.4 Автоматическая генерация тестов из контрактов . . . 40

**7 Мета-уровень: система тестирует себя** **41**
7.1 Рекурсивное тестирование . . . 41
7.2 Иерархия тестирования . . . 43
7.3 Ограничения рекурсивного тестирования . . . 44
7.4 Порождатель как генератор тест-потоков . . . 44

**8 Roadmap** **45**
8.1 MVP (1 день) . . . 45
8.2 Неделя 1 . . . 46
8.3 Неделя 2 . . . 46
8.4 Месяц . . . 47
8.5 Квартал . . . 47

**9 Риски** **48**
9.1 Стоимость и ресурсы . . . 49

**10 Заключение** **49**
10.1 Ключевые выводы . . . 49
10.2 Критерий успеха . . . 50

10.3 Следующие шаги . . . . . 50

# 1. Введение

## 1.1. Постановка задачи

Артемий формулирует задачу:

> *«Просмотреть граф как система сейчас реализована (описанный человеком) и реализовать на основе графа автотесты по системе: входные данные такие-то, целевое состояние должно быть таким-то, оно должно быть достижимо, а нежелательные состояния должны выбрасывать исключения.»*

**Суть задачи** в трёх шагах:

1. Человек описывает систему как **граф состояний/переходов** (текстом, схемой, или формально)
2. Система автоматически **порождает тесты** вида:
   - «При входе $X \to$ целевое состояние $Y$ **достижимо**»
   - «При входе $X' \to$ должно выброситься исключение (нежелательное состояние)»
3. Тесты **запускаются** и проверяют реальную систему

## 1.2. Контекст системы `development_system`

`development_system` — метациркулярная среда вычислительных потоков:

| Компонент | Роль |
|---|---|
| Telegram | Входная точка: пользователь отправляет команды |
| Мастер (SBCL) | Polling loop, маршрутизация команд, хранение истории |
| `команды.lisp` | Обработчики команд (`/запустить`, `/обучить`, ...) |
| `потоки.lisp` | Загрузка и запуск потоков через reflection |
| `мастер/потоки/*.lisp` | Каждый поток: один пакет, одна функция `выполнить` |
| Порождатель | Метапоток: по описанию генерирует новый поток через LLM |

**Ключевые потоки:**

- `эхо` — возвращает входную задачу (для отладки)
- `уточнение` — уточняет задачу через LLM
- `исполнитель` — выполняет задачу через LLM
- `питон` — LLM генерирует Python-код, запускает, возвращает вывод
- `порождатель` — принимает описание на русском, генерирует новый поток

### 1.3. Механизм загрузки потоков

```
(defun найтивыполнить%-имя()
  "Находит функциюВЫПОЛНИТЬвпакетеПОТОКИМЯ    :-"
  (let* ((pkg (find-package
```

```
                (string-upcase (concatenate 'string "поток-" имя))))
          (sym (when pkg (find-symbol "ВЫПОЛНИТЬ" pkg))))
     (when (and sym (fboundp sym))
       (symbol-function sym))))

;; Состояниесистемы
(defparameter активныепотоки*-* (make-hash-table :test #'equal))
```

Listing 1: Reflection-based поиск функции выполнить

**Метациркулярность:** порождатель — сам поток, который генерирует другие потоки через `compile-file` + `load` в живой образ SBCL.

### 1.4. Цель исследования

Спроектировать и реализовать `поток-граф-тестировщик`, который:

1. Принимает описание системы (естественный язык, JSON, Lisp-DSL)
2. Строит внутреннее представление графа состояний
3. Автоматически генерирует тест-кейсы через анализ графа
4. Выполняет тесты через существующую инфраструктуру
5. Возвращает отчёт: passed / failed / unreachable

## 2. Теоретические основы

### 2.1. Model-Based Testing (MBT)

**Model-Based Testing** — методология тестирования, при которой тест-кейсы автоматически генерируются из формальной модели системы.

**Ключевая идея MBT**

Вместо ручного написания тестов инженер создаёт **модель** системы (граф состояний, конечный автомат, спецификацию). Алгоритм обходит модель и генерирует последовательности входов, покрывающие все переходы.

#### 2.1.1. Transition Tree Method

Метод дерева переходов — базовый алгоритм MBT:

1. Начать с начального состояния (корень дерева)
2. Для каждого исходящего перехода создать ветку
3. Рекурсивно повторить для всех достижимых состояний
4. Каждый путь от корня до листа — один тест-кейс

### 2.1.2. Критерии покрытия

| Критерий | Описание | Полнота |
|---|---|---|
| All-states | Каждое состояние посещается хотя бы раз | Низкая |
| All-transitions | Каждый переход (ребро) выполняется хотя бы раз | Средняя |
| All-paths | Все возможные пути от начала до конца | Высокая |

*Для `development_system` рекомендуется:* **All-transitions** — оптимальный баланс между полнотой и трудоёмкостью.

### 2.1.3. Negative Testing через граф

Negative testing — проверка того, что система корректно обрабатывает некорректные входы. В контексте графа:

- Определить **запрещённые состояния** (forbidden states)
- Найти пути, ведущие к ним
- Сгенерировать тесты, ожидающие исключение

## 2.2. Property-Based Testing

**Property-Based Testing (PBT)** — подход, при котором вместо конкретных тест-кейсов формулируются **инварианты** (свойства), которые должны выполняться для любого входа.

```
;; Инвариант: потокэхо- всегдавозвращаетстроку  , равнуювходу
(for-all вход(( (gen-string)))
  (is (string= входзапуститьпоток (- "эхо" вход))))
```

Listing 2: Пример property для потока эхо

### 2.2.1. QuickCheck-подход

Алгоритм QuickCheck:

1. Сгенерировать случайный вход из заданного генератора
2. Выполнить функцию
3. Проверить свойство
4. При неудаче — shrink (минимизировать контрпример)
5. Повторить N раз (обычно 100)

### 2.2.2. Применение к CL-потокам

Для потоков `development_system` можно сформулировать свойства:

| Свойство | Формулировка |
|---|---|
| Типизация | `(выполнить x)` всегда возвращает строку или выбрасывает исключение |
| Терминация | `(выполнить x)` завершается за конечное время (timeout) |
| Идемпотентность | Для `эхо`: повторный вызов даёт тот же результат |
| Детерминизм | Для `эхо`: один вход $\rightarrow$ один выход |

### 2.2.3. CL-библиотеки для PBT

- `fiveam` — популярный testing framework с базовой поддержкой generators
- `cl-quickcheck` — порт Haskell QuickCheck (заброшен, но работает)
- `check-it` — современная альтернатива с shrinking

## 2.3. Reachability Analysis

**Reachability Analysis** — анализ достижимости состояний в графе.

### 2.3.1. Алгоритм

```
(defun достижимыесостояния- граф( начало)
  "Возвращает множествовсехдостижимыхсостоянийизНАЧАЛО     "
  (let посещённые(( (make-hash-table :test #'equal))очередь
        ( (list начало)))
    (loop while очередь
          for текущее = (pop очередь)
          unless (gethash текущееепосещённые )
          do (setf (gethash текущееепосещённые ) t)
             (dolist сосед( соседи( графтекущее ))
               (push соседочередь )))
    (hash-table-keys посещённые)))
```

Listing 3: BFS для анализа достижимости

### 2.3.2. Safety vs Liveness Properties

| Тип | Формулировка | Пример в системе |
|---|---|---|
| Safety | «Плохое никогда не произойдёт» | Поток не зависает бесконечно |
| Liveness | «Хорошее в итоге случится» | Запрос в итоге получит ответ |

### 2.3.3. Нежелательные состояния в `development_system`

- Поток выбросил необработанную ошибку (без `handler-case`)
- Мастер завис (нет ответа > N секунд)
- Порождённый поток не компилируется
- LLM вернул невалидный код
- Бесконечный цикл в потоке

## 2.4. TLA+ и формальная верификация

**TLA+** (Temporal Logic of Actions) — язык формальной спецификации, разработанный Лесли Лэмпортом для описания конкурентных систем.

### 2.4.1. Применимость к CL-потокам

TLA+ описывает систему как:

- **Состояние** — значения всех переменных
- **Действие** — предикат на паре (старое состояние, новое состояние)

- **Спецификация** — формула темпоральной логики

**Ограничения TLA+**

TLA+ требует изоморфного описания системы на отдельном языке. Для `development_system` это означает дублирование логики. Рекомендуется **lightweight-подход**: описание графа без полного model checking.

### 2.5. Alloy для описания графа

**Alloy** — язык декларативного моделирования с автоматическим поиском контрпримеров.

#### 2.5.1. Relational Model

В Alloy всё — отношения:

```
sig State {}
sig Flow { executes: State -> State }

pred valid_flow[f: Flow] {
  all s: State | some f.executes[s] implies
    f.executes[s] in State
}

run valid_flow for 5
```

Listing 4: Пример Alloy-спецификации

### 2.5.2. Применимость

Alloy полезен для:

- Поиска контрпримеров к инвариантам
- Проверки полноты графа (все состояния достижимы)
- Обнаружения тупиков (deadlocks)

*Для MVP:* Alloy избыточен. Достаточно ручного анализа графа с BFS/DFS.

## 2.6. LLM как генератор тест-кейсов

### 2.6.1. Prompt Engineering для извлечения тестов

```
Системаописанаследующимобразом
   :описание
< системы>Графсостояний

 :
- Начальноесостояние : idle
- Целевыесостояния : result_ready
- Запрещённыесостояния : error, timeoutСгенерируйтесткейсы

 - вформате :ТЕСТ
: название<>ВХОД
: входные< данные>ОЖИДАНИЕ
: целевое< состояниеили  exception>
```

Listing 5: Промпт для генерации тестов из описания

#### 2.6.2. Проблема галлюцинаций

LLM может сгенерировать несуществующие переходы или состояния.

**Митигация:**

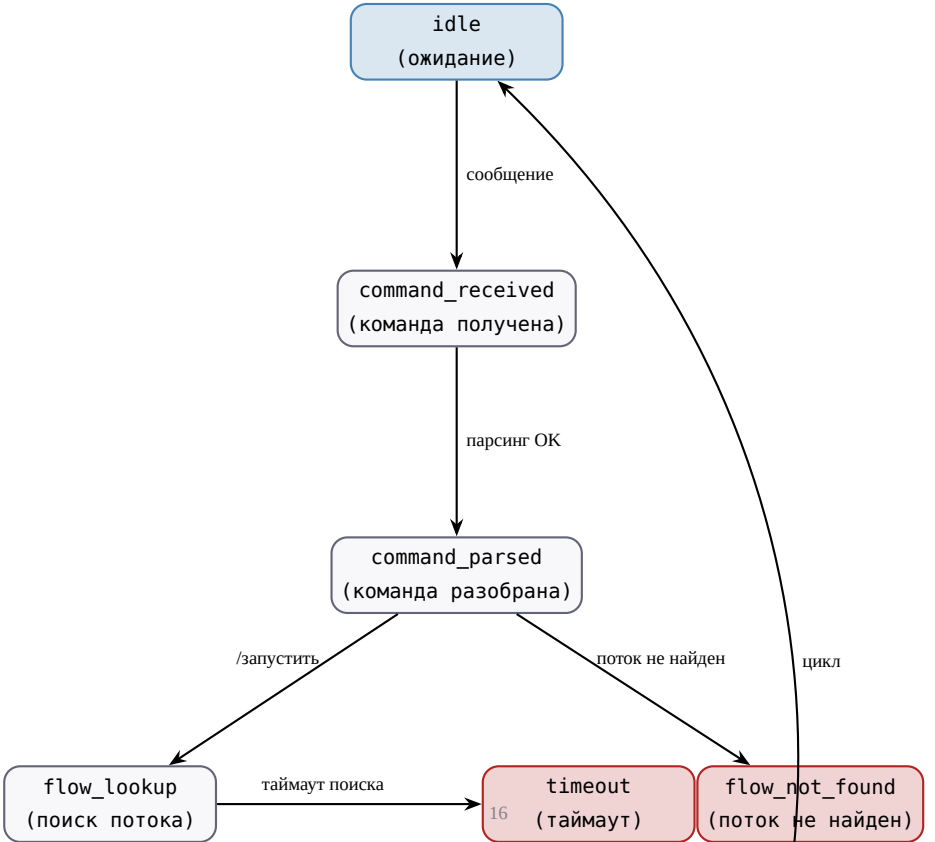
1. Валидация сгенерированных тестов против исходного графа
2. Self-critique: попросить LLM проверить свой же вывод
3. Человек проверяет первые N тестов

## 3. Граф состояний development_system

---

### 3.1. Полный граф цикла обработки сообщения

На рисунке 1 представлен граф состояний для одного цикла обработки входящего сообщения в `development_system`.


idle
(ожидание)
сообщение
command_received
(команда получена)
парсинг OK
command_parsed
(команда разобрана)
/запустить
поток не найден
цикл
flow_lookup
(поиск потока)
таймаут поиска
timeout
(таймаут)
flow_not_found
(поток не найден)

### 3.2. Инварианты системы

| Инвариант | Формулировка |
|---|---|
| I1. Типизация | Функция `выполнить` возвращает строку или выбрасывает `condition` |
| I2. Терминация | Любой вызов завершается за `*таймаут-потока*` секунд |
| I3. Атомарность | Один поток не может изменить состояние другого во время выполнения |
| I4. Идемпотентность | `(выполнить эхо x)` = `(выполнить эхо x)` для любого x |
| I5. Целостность | После ошибки система возвращается в `idle` |

### 3.3. Переходы и триггеры

| Из | В | Триггер |
|---|---|---|
| idle | command_received | Входящее сообщение из Telegram |
| command_received | command_parsed | Успешный парсинг команды |
| command_parsed | flow_lookup | Команда `/запустить <имя>` |
| command_parsed | flow_not_found | Неизвестная команда |
| flow_lookup | flow_running | Пакет и функция найдены |
| flow_lookup | timeout | Поиск превысил таймаут |
| flow_running | result_ready | `выполнить` вернула строку |
| flow_running | execution_error | `выполнить` выбросила condition |
| flow_running | llm_error | LLM-запрос не удался |
| result_ready | response_sent | Telegram API принял ответ |
| response_sent | idle | Цикл завершён |

## 4. Архитектура поток-граф-тестировщик

### 4.1. Обзор

`Поток-граф-тестировщик` — специализированный поток, который:

1. Принимает описание системы в одном из трёх форматов
2. Парсит описание и строит внутренний граф
3. Применяет алгоритмы анализа графа
4. Генерирует тест-кейсы
5. Выполняет тесты через `запустить-поток`
6. Возвращает структурированный отчёт

### 4.2. Три варианта входного формата

#### 4.2.1. Вариант A: естественный язык

```
Система
: Мастерпринимаеткомандузапустить   / эхозадачаОжидаемое
: возвращаетстрокуравнуюзадачеНежелательное
: пустойответ , исключениевпотокеСистема

: Мастерпринимаеткомандузапустить   / питон "1+1"Ожидаемое
: возвращаетстрокусрезультатомвычисленияНежелательное
: ошибкасинтаксиса , таймаут
```

Listing 6: Описание на естественном языке

**Плюсы:** Минимальный порог входа, понятно любому.

**Минусы:** LLM должен интерпретировать, возможны ошибки.

### 4.2.2. Вариант Б: JSON

```
{
  "nodes": [
    {"id": "idle", "type": "initial"},
    {"id": "command_received", "type": "normal"},
    {"id": "flow_running", "type": "normal"},
    {"id": "result_ready", "type": "target"},
    {"id": "error", "type": "forbidden"}
  ],
  "edges": [
    {"from": "idle", "to": "command_received",
     "trigger": "message_received"},
    {"from": "command_received", "to": "flow_running",
     "trigger": "flow_found"},
    {"from": "command_received", "to": "error",
     "trigger": "flow_not_found"},
    {"from": "flow_running", "to": "result_ready",
     "trigger": "execute_ok"},
    {"from": "flow_running", "to": "error",
     "trigger": "exception"}
  ],
  "tests": {
    "happy_path": {
      "input": "запустить/ эхопривет ",
      "target": "result_ready"
    },
    "error_path": {
```

```
      "input": "запустить/ несуществующий",
      "target": "error",
      "expect_exception": true
    }
  }
}
```

Listing 7: Граф в формате JSON

**Плюсы:** Точный, машиночитаемый, нет интерпретации.

**Минусы:** Громоздкий, требует знания формата.

### 4.2.3. Вариант B: Lisp DSL

```
(defgraph мастерцикл-
  ;; Состояния
  (state :idle         :initial t)
  (state :cmd-received)
  (state :flow-running)
  (state :result-ready :target t)
  (state :error        :forbidden t)
  (state :timeout      :forbidden t)

  ;; Переходы
  (transition :idle -> :cmd-received
    :via "сообщение получено")
  (transition :cmd-received -> :flow-running
    :when "поток найден")
  (transition :cmd-received -> :error
```

```
    :when "поток ненайден ")
  (transition :flow-running -> :result-ready
    :when "выполнить вернуластроку ")
  (transition :flow-running -> :error
    :when "exception")
  (transition :flow-running -> :timeout
    :when "превышен таймаут")

  ;; Тесты
  (test "эхо возвращаетвход "
    :input "запустить/ эхопривет "
    :expect :result-ready
    :assert (string= результат "привет"))

  (test "несуществующий поток - ошибка"
    :input "запустить/ несуществующий"
    :expect :error))
```

Listing 8: Граф в Lisp DSL

**Плюсы:** Естественный для системы на Lisp, расширяемый.
**Минусы:** Требует знания Lisp.

**Рекомендация**

Для `development_system` рекомендуется **Lisp DSL** как основной формат. JSON — для интероперабельности. Естественный язык — для быстрых прототипов через LLM.

### 4.3. Алгоритм генерации тестов из графа

1. **Парсинг описания** → внутренний граф (adjacency list)
2. **BFS от начального состояния** до всех target states → happy path tests
3. **BFS до forbidden states** → negative tests (ожидаем exception)
4. **Анализ unreachable states** → dead code / неполная спецификация
5. **Генерация CL-кода** через порождатель → тест-потоки

```
(defun сгенерироватьтесты- граф()
  "Генерирует тесткейсы- изграфасостояний  "
  (let начало(( найтиначальное(- граф))целевые
        ( найтицелевые(- граф))запрещённые
        ( найтизапрещённые(- граф))тесты
        ( nil))
    ;; Happy path: путикцелевымсостояниям
    (dolist цель( целевые)
      (let путь(( (bfsпуть- графначалоцель  )))
        (when путь
          (push (makeтест- имя: (format nil "happy--~A" цель)путь
                            : путьожидание
                            : :success)тесты
                ))))
    ;; Negative: путикзапрещённымсостояниям
    (dolist запр( запрещённые)
      (let путь(( (bfsпуть- графначалозапр  )))
        (when путь
          (push (makeтест- имя: (format nil "negative--~A" запр)путь
```

```
                          : путьожидание
                          : :exception)тесты
                ))))
    ;; Недостижимыесостояния  — предупреждение
    (let достижимые(( достижимыесостояния(— графначало )))
      (dolist узел( всеузлы(— граф))
        (unless (member узелдостижимые )
          (push (макетест— имя: (format nil "unreachable-~A" узел)путь
                          : niloжидание
                          : :warning)тесты
                ))))
    (nreverse тесты)))
```

Listing 9: Алгоритм генерации тестов

### 4.4. Интеграция с существующим мастером

```
;; Запускс  JSONграфом—запустить
/ графтестировщик— граф={"nodes":[...], "edges":[...]}

;; Запускс  DSLзапустить
/ графтестировщик— дсл=(defgraph мастерцикл— ...)

;; Запускcчтениемизфайлазапустить
/ графтестировщик— файл=AGENT.md

;; Результат:
;; тест —1: запустить/ эхопривет "" —> привет"" PASS
;; тест —2: запустить/ питон "1+1" —> "2" PASS
```

```
;; тест -3: запустить/ несуществующий -> ERROR (expected) FAIL
;; тест -4: состояние timeout - недостижимовтестах
```

Listing 10: Пример команд

# 5. Реализация

## 5.1. Структуры данных

```
(defpackage потокграфтестировщик:--
  (:use :cl)
  (:export выполнить#:парситьграф
           #:- сгенерироватьтесты#:-запуститьтесты
           #:- форматироватьотчёт#:-))

(in-package потокграфтестировщик:--)

;;; Узелграфа
(defstruct узел
  (id nil :type symbol)тип
  ( :normal :type (member :initial :normal :target :forbidden))метаданные
  ( nil :type list))

;;; Ребрографа
(defstruct реброиз
  ( nil :type symbol)в
```

```
  ( nil :type symbol)триггер
  ( "" :type string)условие
  ( nil :type (or null function)))

;;; Граф
(defstruct графузлы
  ( nil :type list)    ; списокструктурузелрёбра
  ( nil :type list)   ; списокструктурребротесты
  ( nil :type list))  ; предопределённыетесты

;;; Тесткейс-
(defstruct тесткейс-имя
  ( "" :type string)путь
  ( nil :type list)         ; последовательностьузловвход
  ( "" :type string)        ; входныеданныеожидание
  ( :success :type (member :success :exception :warning))
  (assert nil :type (or null function)))

;;; Результаттеста
(defstruct результаттеста-тест
  ( nil :type тесткейс-)статус
  ( :pending :type (member :pass :fail :skip :error :pending))выход
  ( nil)времямс
  (- 0 :type integer)сообщение
  ( "" :type string))
```

Listing 11: Базовые структуры данных

## 5.2. Парсинг Lisp DSL

```
(defmacro defgraph имя( &body определения)
  "Определяет графсостояний "
  `(progn
     (defparameter имя,
       (makeграф-узлы
        : (list ,@(mapcar обработать#'-state
                          (remove-if-not #'state-p определения)))рёбра
        : (list ,@(mapcar обработать#'-transition
                           (remove-if-not #'transition-p определения)))тесты
        : (list ,@(mapcar обработать#'-test
                           (remove-if-not #'test-p определения)))))имя
     ,))

(defun state-p форма()
  (and (listp форма) (eq (first форма) 'state)))

(defun transition-p форма()
  (and (listp форма) (eq (first форма) 'transition)))

(defun test-p форма()
  (and (listp форма) (eq (first форма) 'test)))

(defun обработать-state форма()
  ;; (state имя: :initial t :target nil :forbidden nil)
  (destructuring-bind (state id &key initial target forbidden) форма
    (declare (ignore state))
    `(makeузел- :id ,idтип
                : ,(cond (initial :initial)
                         (target :target)
```

```
                            (forbidden :forbidden)
                            (t :normal)))))

(defun обработать-transition форма()
  ;; (transition из: -> в: :when условие"" :via триггер"")
  (destructuring-bind (transition из arrow в &key when via) форма
    (declare (ignore transition arrow))
    `(makeребро- из: из, в: в,триггер
                 : ,(or via when ""))))
```

Listing 12: Макрос defgraph

## 5.3. Парсинг JSON

```
(defun парсить-jsonграф- (jsonстрока-)
  "Парсит JSONописание- графа"
  (let* данные(( (yason:parse jsonстрока-))узлы
         (-json (gethash "nodes" данные))рёбра
         (-json (gethash "edges" данные))тесты
         (-json (gethash "tests" данные)))
    (makeграф-узлы
     : (mapcar
            (lambda y()
              (makeузел-
               :id (intern (string-upcase (gethash "id" y)) :keyword)тип
               : (intern (string-upcase
                            (or (gethash "type" y) "normal"))
                           :keyword)))узлы
            -json)рёбра
```

```
    : (mapcar
            (lambda p()
              (makeребро-из
               : (intern (string-upcase (gethash "from" p)) :keyword)в
               : (intern (string-upcase (gethash "to" p)) :keyword)триггер
               : (or (gethash "trigger" p) "")))рёбра
            -json)тесты
    : (when тесты-json
             (maphash
              (lambda имя( спец)
                (makeтесткейс--имя
                 : имявход
                 : (gethash "input" спец)ожидание
                 : (if (gethash "expect_exception" спец)
                             :exception
                             :success)))тесты
              -json)))))
```

Listing 13: Парсинг JSON в граф

## 5.4. Алгоритмы анализа графа

```
;;; Adjacency list изрёбер
(defun построить-adjacency граф()
  (let ((adj (make-hash-table :test #'eq)))
    (dolist ребро( графрёбра(- граф))
      (push ребров(- ребро)
            (gethash реброиз(- ребро) adj)))
    adj))
```

```lisp

;;; BFS поискпути
(defun bfsпуть- граф( изв )
  "Возвращает кратчайшийпутьотИЗдоВили       NIL"
  (let ((adj построить(-adjacency граф))посещённые
        ( (make-hash-table :test #'eq))родители
        ( (make-hash-table :test #'eq))очередь
        ( (list из)))
    (setf (gethash изпосещённые ) t)
    (loop while очередь
          for текущий = (pop очередь)
          do (when (eq текущийв )
               (return-from bfsпуть-восстановитьпуть
                 (- родителиизв  )))
             (dolist сосед( (gethash текущий adj))
               (unless (gethash соседпосещённые )
                 (setf (gethash соседпосещённые ) t)
                 (setf (gethash соседродители ) текущий)
                 (push соседочередь ))))
    nil))

(defun восстановитьпуть- родители( изв )
  (let путь(( (list в)))
    (loop for текущий = (gethash (first путь) родители)
          while (and текущий (not (eq текущийиз )))
          do (push текущийпуть ))
    (push изпуть )путь
    ))

;;; Поискcостоянийпотипу
```

```
(defun найтипотипу-- граф( тип)
  (remove-if-not (lambda у() (eq узелтип(- у) тип))графузлы
                 (- граф)))

(defun найтиначальное- граф()узел
  (-id (first найтипотипу(-- граф :initial))))

(defun найтицелевые- граф()
  (mapcar узел#'-id найтипотипу(-- граф :target)))

(defun найтизапрещённые- граф()
  (mapcar узел#'-id найтипотипу(-- граф :forbidden)))
```

Listing 14: BFS и анализ достижимости

### 5.5. Выполнение тестов

```
(defparameter таймауттеста*-* 10) ; секунды

(defun выполнитьтест- тест()
  "Выполняет одинтесткейс - ивозвращаетрезультат  "
  (let начало(( (get-internal-real-time))выход
        ( nil)статус
        ( :pending)сообщение
        ( ""))
    (handler-case
        (progn
          ;; Парсимвход : запустить/ поток<> аргумент<>
          (multiple-value-bind поток( аргумент)парситькомандутест
```

```
              (-- тесткейсвход(-- тест))
            (setf выходзапуститьпоток (- потокаргумент ))
            ;; Проверяеможидание
            (case тесткейсожидание(-- тест)
              (:success
               (if (stringp выход)
                   (progn
                     (setf статус :pass)
                     (setf сообщение (format nil "OK: ~A"
                                             (subseq выход 0
                                                     (min 50 (length выход))))))
                   (progn
                     (setf статус :fail)
                     (setf сообщение "Ожидалась строка"))))
              (:exception
               ;; Ожидалиисключение , нонеполучили
               (setf статус :fail)
               (setf сообщение "Ожидалось исключение, полученрезультат ")))))
      (error (e)
        ;; Получилиисключение
        (case тесткейсожидание(-- тест)
          (:exception
           (setf статус :pass)
           (setf сообщение (format nil "OK: exception ~A" (type-of e))))
          (otherwise
           (setf статус :error)
           (setf сообщение (format nil "ERROR: ~A" e))))))
    (let конец(( (get-internal-real-time)))
      (makeрезультаттеста--тест
       : тестстатус
```

```
      : статусвыход
      : выходвремямс
      :- (round (* 1000 (/ (- конецначало )
                                internal-time-units-per-second)))сообщение
      : сообщение))))

(defun запуститьтесты- граф()
  "Выполняет всетестыизграфа   "
  (let тесты(( (append графтесты(- граф)сгенерироватьтесты
                       (- граф))))
    (mapcar выполнитьтест#'- тесты)))
```

Listing 15: Выполнение тест-кейсов

### 5.6. Точка входа потока

```
(defun выполнитьзадача ()
  "Главная точкавходапотокаграфтестировщика  --"
  (handler-case
      (let* формат(( определитьформат(- задача))граф
             ( (case формат
                     (:json парсить(-jsonграф- извлечь(-json задача)))
                     (:dsl  (eval (read-from-stringизвлечь
                                  (-dsl задача))))
                     (:file парситьфайл(- извлечьпуть(- задача)))
                     (t парситьестественный    (- задача))))результаты
             ( запуститьтесты(- граф)))форматироватьотчёт
        (- результаты))
    (error (e)
```

```
      (format nil "□ Ошибкаграфтестировщика -: ~A" e))))

(defun определитьформат- задача()
  (cond ((search "граф={" задача) :json)
        ((search "дсл=(" задача) :dsl)
        ((search "файл=" задача) :file)
        (t естественный:)))

(defun форматироватьотчёт- результаты()
  (with-output-to-string (out)
    (format out "□~% Отчёттестирования ~%")
    (format out "━━━
```

```
      (format out "━━━
```

```
  (is (string= "привет"запуститьпоток
               (- "эхо" "привет")))
  (is (string= ""запуститьпоток
               (- "эхо" "")))
  (is (string= "123"запуститьпоток
               (- "эхо" "123"))))

(test несуществующийпотокошибка--
  "Несуществующий потокдолженвыброситьошибку   "
  (signals errorзапуститьпоток
    (- "несуществующийпоток--xyz" "тест")))

(test потоквозвращаетстроку--
  "Любой потокдолженвернутьстрокуиливыброситьошибку      "
  (dolist имя( '("эхо" "уточнение" "исполнитель"))
    (let результат(( запуститьпоток(- имя "тест")))
      (is (stringp результат)
          "Поток ~A долженвернутьстроку  " имя))))
```

Listing 17: Настройка fiveam для потоков

## 6.2. Property-Based Testing с check-it

```
(ql:quickload :check-it)

(defpackage тестысвойств:-
  (:use :cl :check-it)
  (:export проверитьсвойства#:-))

```

```
(in-package тестысвойств:-)

;;; Генераторы
(defgenerator gen-safe-string ()
  "Генератор безопасныхстрокбез  ( управляющихсимволов )"
  (let ((len (random 100)))
    (coerce (loop repeat len
                  collect (code-char (+ 32 (random 95))))
            'string)))

;;; Свойства
(defproperty эхоидемпотентен-
  "Эхо всегдавозвращаеттожесамоеприповторномвызове       "вход
  (( (gen-safe-string)))
  (let результат((-1 запуститьпоток(- "эхо" вход))результат
        (-2 запуститьпоток(- "эхо" вход)))
    (string= результат-1 результат-2)))

(defproperty эхо-identity
  "Эхо является identityфункцией-"вход
  (( (gen-safe-string)))
  (string= входзапуститьпоток (- "эхо" вход)))

(defproperty всепотокивозвращаютстроку---
  "Любой потоквозвращаетстрокуиливыбрасываетошибку     "имя
  (( (gen-element '("эхо" "уточнение")))вход
   ( (gen-safe-string)))
  (handler-case
      (stringp запуститьпоток(- имявход ))
    (error () t)))  ; исключение - тожедопустимыйрезультат
```

```

;;; Запуск
(defun проверитьсвойства- ()
  (check эхоидемпотентен- :times 100)
  (check эхо-identity :times 100)
  (check всепотокивозвращаютстроку--- :times 50))
```

Listing 18: Property-based тесты

### 6.3. Contract-Based Testing

```
(defpackage контракты:
  (:use :cl)
  (:export #:defcontract #:with-contract проверитьконтракт#:-))

(in-package контракты:)

;;; Структураконтракта
(defstruct контрактимя
  ( nil :type symbol)предусловие
  ( nil :type function)постусловие
  ( nil :type function)инвариант
  ( nil :type (or null function)))

;;; Хранилищеконтрактов
(defparameter контракты** (make-hash-table :test #'equal))

;;; Макросдляопределенияконтракта
(defmacro defcontract имяпотока(- &key pre post invariant)
```

```
  "Определяет контрактдляпотока  "
  `(setf (gethash имяпотока,- контракты**)
         (makeконтракт-имя
          : (intern (string-upcase имяпотока,-))предусловие
          : (lambda вход() ,pre)постусловие
          : (lambda вход( выход) ,post)инвариант
          : ,(when invariant
                        `(lambda () ,invariant)))))

;;; Примерконтрактов
(defcontract "эхо"
  :pre (stringp вход)
  :post (and (stringp выход)
             (string= входвыход )))

(defcontract "питон"
  :pre (stringp вход)
  :post (stringp выход)
  :invariant (< (length выход) 100000))  ; защитаотбесконечноговывода

;;; Обёрткаспроверкойконтракта
(defun сконтрактом- имяпотока(- вход)
  "Выполняет потокспроверкойконтракта   "
  (let контракт(( (gethash имяпотока- контракты**)))
    (when контракт
      ;; Проверяемпредусловие
      (unless (funcall контрактпредусловие(- контракт) вход)
        (error "Нарушено предусловиедля  ~A: ~A" имяпотока- вход)))
    ;; Выполняем
    (let выход(( запуститьпоток(- имяпотока- вход)))
```

```
      (when контракт
        ;; Проверяемпостусловие
        (unless (funcall контрактпостусловие(- контракт) входвыход )
          (error "Нарушено постусловиедля  ~A" имяпотока-))
        ;; Проверяеминвариант
        (when контрактинвариант(- контракт)
          (unless (funcall контрактинвариант(- контракт))
            (error "Нарушен инвариантдля  ~A" имяпотока-))))выход
      )))
```

Listing 19: Контракты для потоков

## 6.4. Автоматическая генерация тестов из контрактов

```
(defun сгенерироватьтестыизконтрактов--- ()
  "Генерирует тесткейсы- извсехзарегистрированныхконтрактов   "
  (let тесты(( nil))
    (maphash
     (lambda имя( контракт)
       ;; Тест: предусловиевыполняетсядлявалидноговхода
       (push (maketesткейс--имя
              : (format nil "~A-pre-valid" имя)вход
              : "тестовая строка"ожидание
              : :success)тесты
             )
       ;; Тест: постусловиевыполняется
       (push (maketesткейс--имя
              : (format nil "~A-post-holds" имя)вход
              : "тест"ожидание
```

```
              : :success
              :assert (lambda вход( выход)
                        (funcall контрактпостусловие(- контракт)входвыход
                                  )))тесты
            ))контракты
     **)
    (nreverse тесты)))
```

Listing 20: Генерация тестов из контрактов

## 7. Мета-уровень: система тестирует себя

### 7.1. Рекурсивное тестирование

`Граф-тестировщик` сам является потоком в системе. Это создаёт интересную возможность: **система может тестировать себя**.

```
;; Графдлятестированиясамогографтестировщика    -
(defgraph тестировщикмета-
  (state :idle :initial t)
  (state :parsing)
  (state :generating)
  (state :executing)
  (state :reporting :target t)
  (state :parse-error :forbidden t)
  (state :execution-error :forbidden t)
```

```
10
11  (transition :idle -> :parsing
12    :via "получен граф")
13  (transition :parsing -> :generating
14    :when "граф валиден")
15  (transition :parsing -> :parse-error
16    :when "синтаксическая ошибка")
17  (transition :generating -> :executing
18    :when "тесты сгенерированы")
19  (transition :executing -> :reporting
20    :when "все тестывыполнены ")
21  (transition :executing -> :execution-error
22    :when "критическая ошибка")
23
24  (test "самотест: пустойграф "
25    :input "запустить/ графтестировщик- граф={}"
26    :expect :parse-error)
27
28  (test "самотест: простойграф "
29    :input "запустить/ графтестировщик- граф={\"nodes\":[{\"id\":\"a\",\"type\":\"initial\"}],\"edges\":[]}"
30    :expect :reporting))
```

Listing 21: Самотестирование графа-тестировщика

### 7.2. Иерархия тестирования

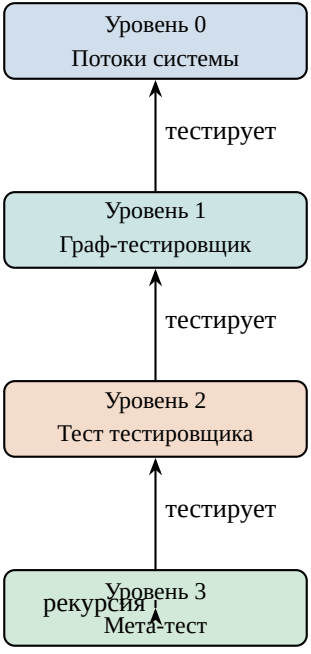



Рис. 2: Иерархия мета-тестирования

### 7.3. Ограничения рекурсивного тестирования

**Проблема полноты**

Система не может полностью протестировать себя — это следует из теоремы Гёделя о неполноте. Тест, проверяющий корректность всех тестов, не может быть частью той же системы.

**Практические ограничения:**

1. **Бесконечная регрессия:** Мета-тест тоже нужно тестировать
2. **Состояние образа:** Тесты могут менять глобальное состояние SBCL
3. **Побочные эффекты:** LLM-запросы, файловая система
4. **Детерминизм:** LLM-потоки недетерминированы

**Митигации:**

1. Ограничить глубину мета-тестирования (2–3 уровня)
2. Изолировать тесты через `let` и `unwind-protect`
3. Мокировать LLM-запросы для детерминизма
4. Использовать `with-compilation-unit` для изоляции компиляции

### 7.4. Порождатель как генератор тест-потоков

```
(defun породитьтестпоток-- имятеста(- спецификация)
  "Использует порождательдлясозданияспециализированноготестпотока    -"
  (let описание((
```

```
        (format nil
                "Создай потоктест --~A.Задачапотока

                  : выполнитьтестсогласноспецификации   :
                 ~АПотокдолжен

                  :
                 1. Вызватьтестируемуюфункцию
                 2. Проверитьрезультат
                 3. Вернуть 'PASS' или 'FAIL: причина'"имятеста
                -спецификация
                )))запуститьпоток
    (- "порождатель" описание)))

;; Примериспользованияпородитьтестпоток
(--
 "эходлиннаястрока--"
 "Проверить, чтопотокэхокорректнообрабатываетстрокидлиной      10000 символов")
```

Listing 22: Порождение тест-потока через порождатель

## 8. Roadmap

### 8.1. MVP (1 день)

✓ Файл `тесты.lisp` — наполнить базовыми тест-кейсами

✓ Простые assert-тесты для существующих потоков

✓ Интеграция с fiveam

```
;; Минимальныйнабортестов
(assert (stringp запуститьпоток(- "эхо" "тест")))
(assert (string= "привет" запуститьпоток(- "эхо" "привет")))
(assert (handler-case
            (progn запуститьпоток(- "несуществующий" "") nil)
          (error () t)))
```

Listing 23: MVP: базовые тесты в тесты.lisp

### 8.2. Неделя 1

- `поток-граф-тестировщик.lisp` — принимает JSON-граф
- Парсинг JSON в структуру графа
- BFS для анализа достижимости
- Генерация и выполнение тестов
- Форматирование отчёта для Telegram

### 8.3. Неделя 2

- Lisp DSL (`defgraph`)
- Интеграция с порождателем: автогенерация тест-потоков
- LLM читает AGENT.md → строит граф → генерирует тесты

- Визуализация графа (TikZ / GraphViz экспорт)

### 8.4. Месяц

- Property-based тестирование через check-it
- Контракты: предусловия, постусловия, инварианты
- Метаданные в `.мета` файлах потоков
- CI-подобный цикл: граф-тестировщик запускается при изменениях
- Статистика покрытия переходов

### 8.5. Квартал

- Интеграция с TLA+ для формальной верификации критических потоков
- Mutation testing: автоматические мутации кода для проверки тестов
- Регрессионное тестирование с историей результатов
- Web-интерфейс для просмотра графов и результатов

## 9. Риски

---

| Риск | Описание | Митигация |
|---|---|---|
| Галлюцинации LLM | LLM генерирует несуществующие переходы или состояния | Валидация против исходного графа |
| Изоляция состояния | Тесты влияют на глобальное состояние SBCL | `unwind-protect`, fresh binding |
| Рекурсивное тестирование | Бесконечная регрессия мета-тестов | Ограничение глубины (2–3 уровня) |
| Неполнота спецификации | Граф не описывает все состояния системы | Анализ unreachable + warning |
| Недетерминизм LLM | Результаты LLM-потоков нестабильны | Мокирование для тестов, retry logic |
| Производительность | Много LLM-запросов при генерации тестов | Кэширование, batch, дешёвые модели |

### 9.1. Стоимость и ресурсы

| Операция | Токены | Примерная стоимость |
|---|---|---|
| Парсинг естественного языка в граф | ∼1000 | $0.003 |
| Генерация 10 тест-кейсов | ∼2000 | $0.006 |
| Выполнение теста с LLM-потоком | ∼500 | $0.0015 |
| Полный цикл (10 тестов) | ∼7000 | $0.02 |

Стоимость приемлема для персонального использования.

## 10. Заключение

---

### 10.1. Ключевые выводы

1. **Граф состояний — мощный инструмент.** Формальное описание системы как графа позволяет автоматически генерировать тесты, анализировать достижимость и выявлять мёртвый код.
2. **Три формата — для разных нужд.** Естественный язык для быстрых прототипов, JSON для интероперабельности, Lisp DSL для полного контроля.
3. **MBT + PBT = полное покрытие.** Model-Based Testing покрывает структуру (переходы графа), Property-Based Testing покрывает данные (инварианты).

4. **Метациркулярность — преимущество и сложность.** Система может тестировать себя, но это создаёт проблемы рекурсии и изоляции.
5. **Контракты формализуют ожидания.** Явные предусловия и постусловия превращают неявные предположения в проверяемые свойства.

### 10.2. Критерий успеха

**Через месяц система считается успешной, если:**

- Можно описать новый поток как граф и сразу получить тесты
- Все существующие потоки покрыты хотя бы базовыми тестами
- Граф-тестировщик ловит регрессии при изменениях в потоках
- Время на ручное тестирование сократилось на 50%+

### 10.3. Следующие шаги

1. Реализовать MVP: базовые тесты в `тесты.lisp` + fiveam
2. Описать граф `development_system` в Lisp DSL
3. Создать `поток-граф-тестировщик.lisp`
4. Запустить первый цикл автогенерации тестов
5. Интегрировать с порождателем для автоматического обновления тестов

Документ подготовлен для Артемия. Версия 1.0, 14 марта 2026. Проект: `development_system`.